

Box-Cox and Yeo-Johnson Transformations

What are Power Transformations?

Power transformations are preprocessing techniques used in Machine Learning to make data more **normally distributed**, reduce **skewness**, and stabilize **variance**.

They are commonly used before applying algorithms that assume normality, such as:

- Linear Regression
 - Logistic Regression
 - PCA
 - LDA
-

Why Use Power Transformations?

Real-world data often contains:

- Right-skewed distributions
- Outliers
- Non-constant variance (heteroscedasticity)

Power transformations help to:

- Reduce skewness
 - Stabilize variance
 - Improve model performance
 - Make data closer to a normal distribution
-

1. Box-Cox Transformation

The **Box-Cox transformation** works only with **positive values**.

Condition

$$x > 0$$

Formula

For a parameter λ :

$$y(\lambda) = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(x), & \lambda = 0 \end{cases}$$

Where:

- x = original value
- λ = transformation parameter selected automatically

Advantages

- Effective for highly skewed positive data
- Improves normality
- Stabilizes variance

Limitations

- Cannot handle zero values
- Cannot handle negative values

2. Yeo-Johnson Transformation

The **Yeo-Johnson transformation** is an extension of Box-Cox.

It can handle:

- Positive values
- Zero values
- Negative values

Condition

$x \in (-\infty, +\infty)$

Formula

For $x \geq 0$:

$$y(\lambda) = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(x+1), & \lambda = 0 \end{cases}$$

For $x < 0$:

$$y(\lambda) = \begin{cases} -\frac{(-x+1)^{2-\lambda} - 1}{2-\lambda}, & \lambda \neq 2 \\ -\ln(-x+1), & \lambda = 2 \end{cases}$$

Comparison Table

Feature	Box-Cox	Yeo-Johnson
Positive values	✓	✓

Feature	Box-Cox	Yeo-Johnson
Zero values	✗	✓
Negative values	✗	✓
Reduces skewness	✓	✓
Stabilizes variance	✓	✓
More flexible	✗	✓

Rule of Thumb

Contains zero or negative values?

Yes → Use Yeo-Johnson

No → Use Box-Cox

Scikit-Learn Implementation

```
from sklearn.preprocessing import PowerTransformer

# Box-Cox
pt_boxcox = PowerTransformer(method='box-cox')

# Yeo-Johnson
pt_yeojohnson = PowerTransformer(method='yeo-johnson')

X_transformed = pt_yeojohnson.fit_transform(X)
```

Example

```
import pandas as pd
from sklearn.preprocessing import PowerTransformer

df = pd.DataFrame({
    'income': [1000, 2000, 5000, 10000, 50000]
})

pt = PowerTransformer(method='box-cox')

df['income_transformed'] = pt.fit_transform(df[['income']])
```

Key Points

- **Box-Cox** works only with strictly positive values.
- **Yeo-Johnson** works with positive, zero, and negative values.
- Both transformations reduce skewness and stabilize variance.
- If your data contains zero or negative values, use **Yeo-Johnson**.
- If all values are positive, **Box-Cox** is a good choice.